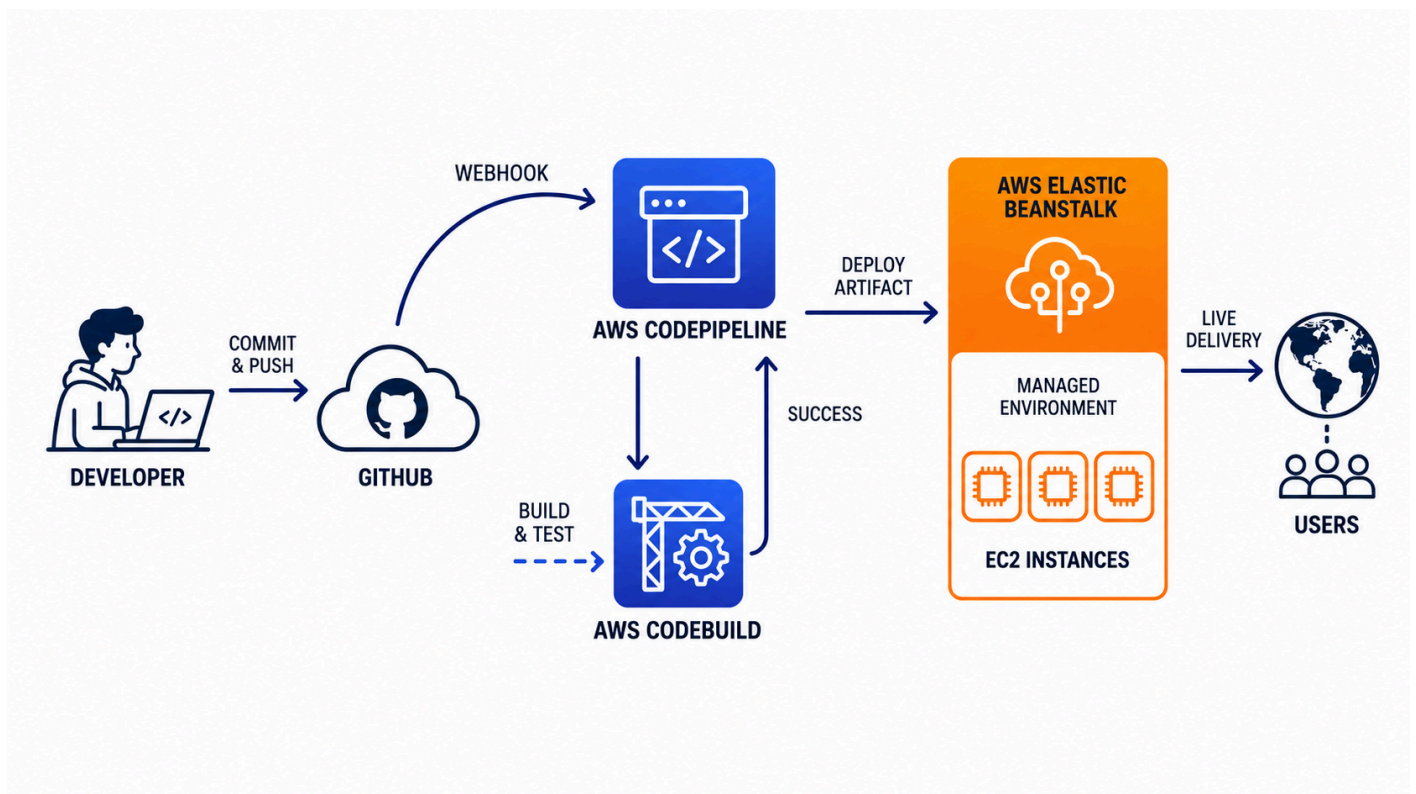


Infrastructure Automation: End-to-End CI/CD Pipeline

Objective: To architect a Continuous Integration and Continuous Deployment (CI/CD) pipeline that automates testing and server provisioning for a Node.js web application so as to minimize significant bottlenecks during the software development lifecycle.

Solution: Engineered an automated release pipeline using AWS CodePipeline. Source code changes pushed to GitHub automatically trigger an AWS CodeBuild environment, which runs unit tests (text validation via **grep**). Upon a successful build, the application artifacts are isolated and deployed to a managed AWS Elastic Beanstalk environment, achieving fully automated continuous delivery.



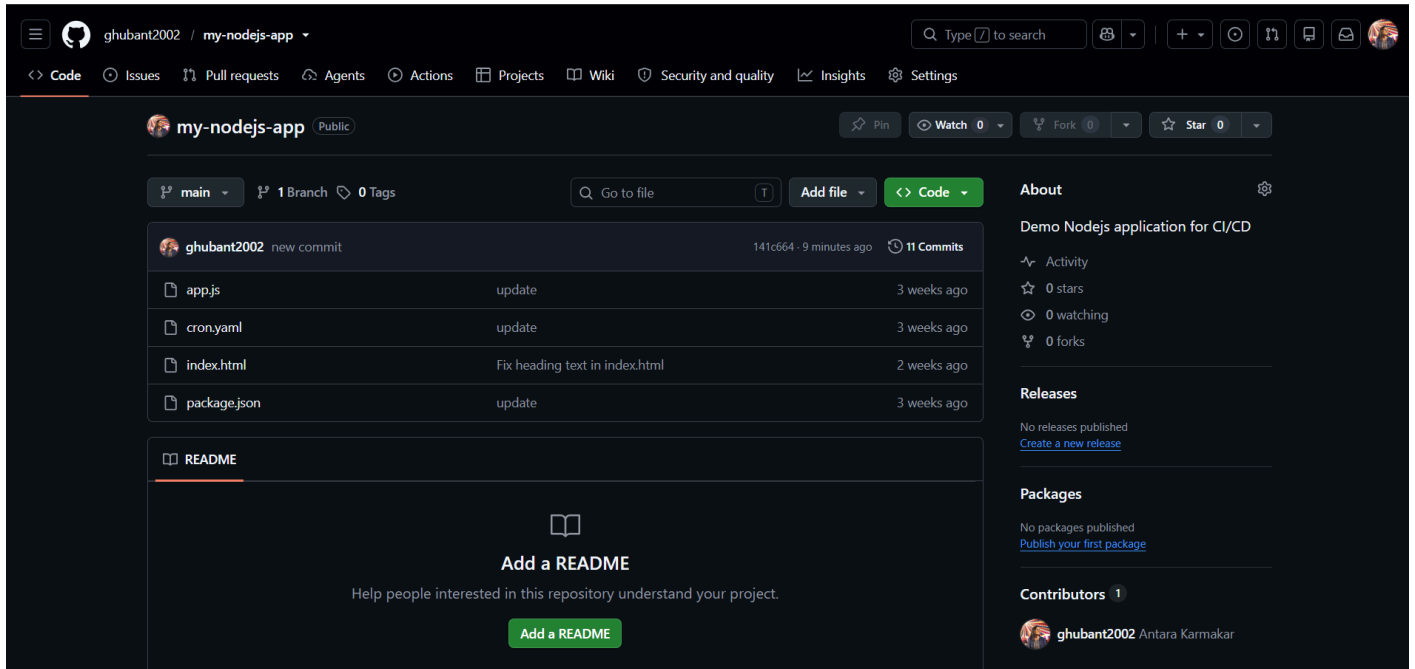
Phase 1: Source Control Initialization (GitHub)

The application code is pushed to a GitHub repository. This serves as the source control for the Frontend (**my-nodejs-app**) application.

Repository Details:

- Repository name: my-nodejs-app
- Description: Demo Nodejs application for CI/CD

The repository is cloned to the local machine (personal setup) in a specific folder where required files are added and then pushed to the GitHub repository. This is done using **GitHub desktop**.



GitHub Repository: Frontend Application Source Code

Phase 2: Continuous Integration (AWS CodeBuild)

CodeBuild compiles the source code, runs unit tests, and produces ready-to-deploy software packages.

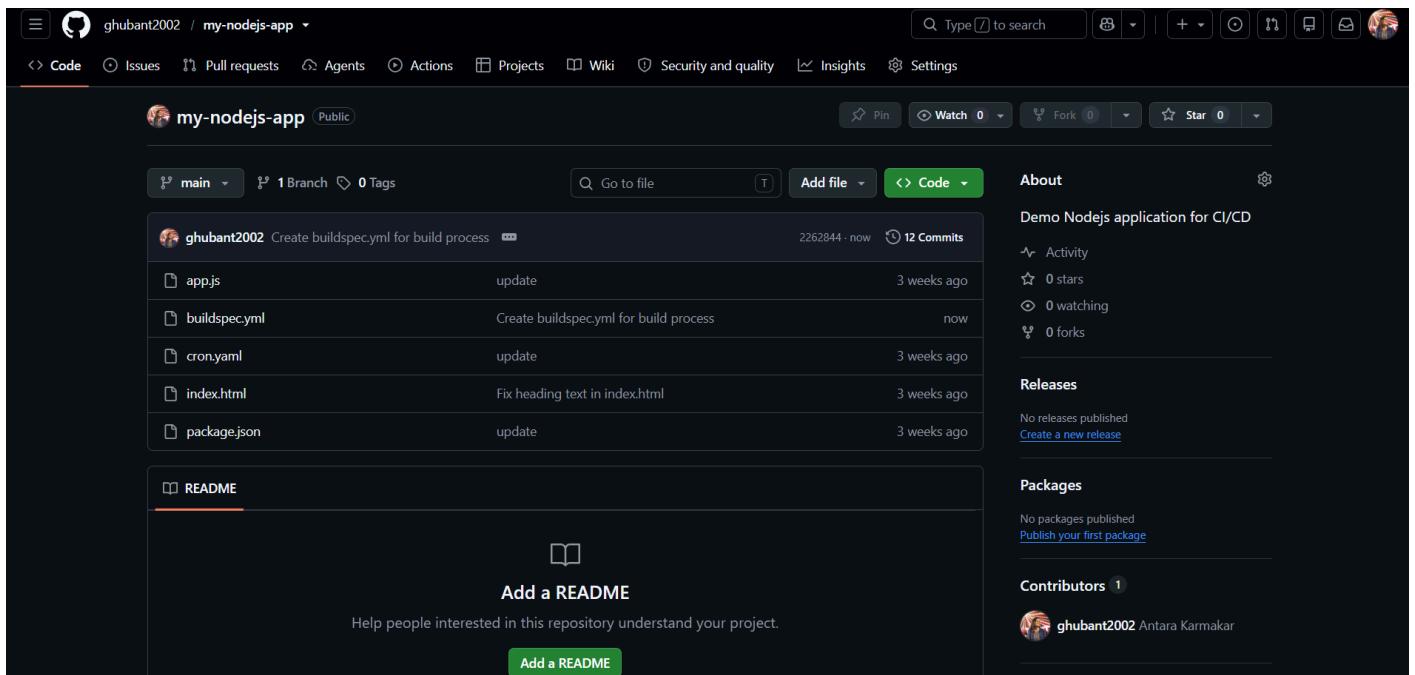
1) Prerequisites:

- The **buildspec.yml** file is created which is a mandatory YAML configuration file used by CodeBuild to execute automated build, test, and packaging commands.

```
! buildspec.yml X
C: > Users > Lenovo > Desktop > antara > AWS_Dev_Associate_Resources > aws-cicd > codebuild > ! buildspec.yml
1  version: 0.2
2
3  phases:
4    install:
5      runtime-versions:
6        nodejs: latest
7      commands:
8        - echo "installing something"
9    pre_build:
10     commands:
11       - echo "we are in the pre build phase"
12    build:
13     commands:
14       - echo "we are in the build block"
15       - echo "we will run some tests"
16       - grep -Fq "Congratulations" index.html
17    post_build:
18     commands:
19       - echo "we are in the post build phase"
20
21  artifacts:
22    files:
23     - index.html
24     - app.js
25     - package.json
26     - cron.yaml
```

YAML Configuration File: buildspec.yml

- The **artifacts** block is added in this file which explicitly tells AWS to package only the application files for the deployment stage, leaving the buildspec behind.
- This file (**buildspec.yml**) is then added to the GitHub repository (**my-nodejs-app**)



GitHub Repository: Added buildspec.yml in the Source Code

2) CodeBuild Configuration:

- Project name: spa-build-test
- Project type: Default project
- Source provider: GitHub
- Connection: create a new OAuth app token connection
- Repository: Repository in my GitHub account
- GitHub repository: <https://github.com/ghubant2002/my-nodejs-app>
- Webhook: Rebuild every time a code change is pushed to this repository (enable)
- Build type: Single build
- Comment Approval: ALL_PULL_REQUESTS (default)
- Approver roles: GITHUB_WRITE, GITHUB_MAINTAIN, GITHUB_ADMIN (default)
- Filter group 1:
 - Event type: PUSH
- Environment:
 - Provisioning model: On-demand (default)
 - Environment image: Managed image (default)
 - Compute: EC2 (default)
 - Running mode: Container (default)
 - Operating system: Ubuntu

- Runtime(s): Standard (default)
- Image: aws/codebuild/standard:8.0 (default)
- Image version: Always use the latest image for this runtime version (default)
- Service role: New service role (default)
- Role name: codebuild-spa-build-test-service-role
- Build specifications: Use a buildspec file

Create build project

Project configuration

Project name
spa-build-test
A project name must be 2 to 255 characters. It can include the letters A-Z and a-z, the numbers 0-9, and the special characters - and _.

Project type
Select what type of project you would like to create. [Info](#)

☒ **Default project**
Create a custom CodeBuild project.

☐ **Runner project**
Create a CodeBuild managed runner for workflows in GitHub Actions, GitHub Enterprise Actions, GitLab, or Buildkite.

Additional configuration
Description, public build access, build badge, concurrent build limit, tags

Source [Add source](#)

Source 1 - Primary
Source provider
GitHub

CodeBuild Configuration: Project Name, Type & Source Provider

Credential
You have not connected to GitHub. Manage account credentials.
☒ Use override credentials for this project only

Connection
You can create a new GitHub connection by using an AWS managed GitHub App, create a new OAuth app token connection, or create a new personal access token connection.
arn:aws:secretsmanager:ap-south-1:426271381760:secret:GitHubLinkCodeBuildNew-LWsOna [G](#)
☒ Only show secrets with tag codebuild:source

Repository
☒ Repository in my GitHub account ☐ Public repository ☐ GitHub scoped webhook

GitHub repository
https://github.com/ghubant2002/my-nodejs-app
https://github.com/<user-name>/<repository-name>

Source version - optional [Info](#)
Enter a pull request, branch, commit ID, tag, or reference and a commit ID.

Additional configuration
Git clone depth, Git submodules, Build status config

CodeBuild Configuration: OAuth App Token Connection & GitHub Repository

Developer Tools > CodeBuild > Build projects > Create build project

▼ Primary source webhook events Info

Webhook - optional Info

☒ Rebuild every time a code change is pushed to this repository

Build type

☒ Single build
Triggers single build

☐ Batch build
Triggers multiple builds as single execution

Comment approval

ALL_PULL_REQUESTS

Approver roles

GITHUB_WRITE X GITHUB_MAINTAIN X GITHUB_ADMIN X

▼ Webhook event filter groups

A build is triggered if any filter group evaluates to true, which occurs when all the filters in the group evaluate to true.

[Add filter group](#)

Filter group 1 [Remove filter group](#)

Event type - optional

Add one or more webhook event filter groups to specify which events trigger a new build. If you do not add a webhook event filter group, then a new build is triggered every time a code change is pushed to your repository.

PUSH X

CodeBuild Configuration: Webhook Filter Group & Build Details

Developer Tools > CodeBuild > Build projects > Create build project

▼ Environment

Provisioning model Info

☒ On-demand
Automatically provision build infrastructure in response to new builds.

☐ Reserved capacity
Use a dedicated fleet of instances for builds. A fleet's compute and environment type will be used for the project.

Environment image

☒ Managed image
Use an image managed by AWS CodeBuild

☐ Custom image
Specify a Docker image

Compute

☒ EC2
Optimized for flexibility during action runs

☐ Lambda
Optimized for speed and minimizes the start up time of workflow actions

Running mode

☒ Container
Running on Docker container

☐ Instance
Running on EC2 instance directly

CodeBuild Configuration: Provisioning Model, Environment Image, Compute & Running Mode

- Webhook is enabled to rebuild the code whenever changes are pushed to the GitHub repository
- Environment settings are configured as per requirements

aws Developer Tools > CodeBuild > Build projects > Create build project

Operating system
Ubuntu

Runtime(s)
Standard

Image
aws/codebuild/standard:8.0

Image version
Always use the latest image for this runtime version

Service role
☒ New service role
Create a service role in your account
☐ Existing service role
Choose an existing service role from your account

Role name
codebuild-spa-build-test-service-role
Type your service role name

► Additional configuration
Timeout, privileged, certificate, VPC, compute type, environment variables, file systems, auto-retry, registry credential

CodeBuild Configuration: Operating System, Runtime, Image & Service Role

aws Developer Tools > CodeBuild > Build projects > Create build project

▼ Buildspec
Build specifications
☐ Insert build commands
Store build commands as build project configuration
☒ Use a buildspec file
Store build commands in a YAML-formatted buildspec file

Buildspec name - optional
By default, CodeBuild looks for a file named buildspec.yml in the source code root directory. If your buildspec file uses a different name or location, enter its path from the source root here (for example, buildspec-two.yml or configuration/buildspec.yml).
buildspec.yml

▼ Batch configuration
You can run a group of builds as a single execution. Batch configuration is also available in advanced option when starting build.
☐ Define batch configuration - optional
You can also define or override batch configuration when starting a build batch.

CodeBuild Configuration: Build Specifications

- The Operating System is set to **Ubuntu** because it provides a highly compatible environment for web development and containerized applications.

The screenshot shows the AWS CodeBuild console interface. The left sidebar contains navigation options under 'Developer Tools' and 'CodeBuild', including 'Source', 'Artifacts', 'Build', 'Build projects', 'Build project', 'Settings', 'Build history', 'Report groups', 'Report history', 'Compute fleets', 'Account metrics', 'Related integrations', 'Jenkins', 'GitHub Actions', and 'GitLab runners'. The main content area displays the 'Build status' for a specific build. The status is 'Succeeded'. Key details include: Initiator: antadmin, Build ARN: arn:aws:codebuild:ap-south-1:426271381760:build/spa-build-test:4c1fb0d4-ae0f-4bae-93e1-45ddbca1a0, Resolved source version: 22628442048b0fc9f46ccfd398ff63f6515b9eea, Start time: Jun 25, 2026 11:38 AM (UTC+5:30), End time: Jun 25, 2026 11:38 AM (UTC+5:30), and Build number: 1. Below this, there are tabs for 'Build logs', 'Phase details', 'Reports', 'Environment variables', 'Build details', and 'Resource utilization'. The 'Phase details' tab is active, showing a table of build phases.

Name	Status	Context	Duration	Start time	End time
SUBMITTED	✓ Succeeded	-	<1 sec	Jun 25, 2026 11:38 AM (UTC+5:30)	Jun 25, 2026 11:38 AM (UTC+5:30)
QUEUED	✓ Succeeded	-	<1 sec	Jun 25, 2026 11:38 AM (UTC+5:30)	Jun 25, 2026 11:38 AM (UTC+5:30)
PROVISIONING	✓ Succeeded	-	5 secs	Jun 25, 2026 11:38 AM (UTC+5:30)	Jun 25, 2026 11:38 AM (UTC+5:30)
DOWNLOAD_SOURCE	✓ Succeeded	-	10 secs	Jun 25, 2026 11:38 AM (UTC+5:30)	Jun 25, 2026 11:38 AM (UTC+5:30)
INSTALL	✓ Succeeded	-	<1 sec	Jun 25, 2026 11:38 AM (UTC+5:30)	Jun 25, 2026 11:38 AM (UTC+5:30)
PRE_BUILD	✓ Succeeded	-	<1 sec	Jun 25, 2026 11:38 AM (UTC+5:30)	Jun 25, 2026 11:38 AM (UTC+5:30)
BUILD	✓ Succeeded	-	<1 sec	Jun 25, 2026 11:38 AM (UTC+5:30)	Jun 25, 2026 11:38 AM (UTC+5:30)

CodeBuild Console: Build Status

Phase 3: Application Deployment (Elastic Beanstalk)

The server environments are configured using Elastic Beanstalk where the application is deployed.

Beanstalk Configuration:

- Application information:
 - Application name: MyWebApp
 - Description: Custom Nodejs Web Application
- Dev Environment under MyWebApp:
 - Environment tier: Web server environment
 - Environment name: MyWebApp-dev-env
 - Platform: Nodejs
 - Platform branch: Node.js 24 running on 64bit Amazon Linux 2023
 - Platform version: 6.11.2 (Recommended)
 - Application code: Upload your code
 - Local file: nodejs-v2-blue.zip
 - Version label: nodejs-v2-blue-dev
 - Configuration presets: Single instance (free tier eligible)
 - Service role: aws-elasticbeanstalk-service-role
 - EC2 instance profile: aws-elasticbeanstalk-ec2-role
- Prod Environment under MyWebApp:
 - Environment tier: Web server environment
 - Environment name: MyWebApp-prod-env
 - Platform: Nodejs

- Platform branch: Node.js 24 running on 64bit Amazon Linux 2023
- Platform version: 6.11.2 (Recommended)
- Application code: Upload your code
- Local file: nodejs-v2-blue.zip
- Version label: nodejs-v2-blue-prod
- Configuration presets: Single instance (free tier eligible)
- Service role: aws-elasticbeanstalk-service-role
- EC2 instance profile: aws-elasticbeanstalk-ec2-role

The screenshot shows the 'Create new application' page in the AWS Elastic Beanstalk console. The left sidebar contains a navigation menu with 'Applications', 'Environments', and 'Change history'. Below this is a 'Recent environments' section listing several environments. The main content area is titled 'Create new application' and includes an 'Info' icon. It contains three sections: 'Application information' with a text input for 'Application name' (containing 'MyWebApp') and a description box (containing 'Custom Nodejs Web Application'); 'Tags' section with a note that no tags are currently associated and an 'Add new tag' button; and 'Cancel' and 'Create' buttons at the bottom right.

Beanstalk Configuration: Application Name & Description

The screenshot shows the 'Configure environment' page in the AWS Elastic Beanstalk console. The left sidebar is the same as the previous screenshot. The main content area is titled 'Configure environment' and includes an 'Info' icon. It features a progress bar on the left with six steps: 'Step 1: Configure environment' (selected), 'Step 2: Configure service access', 'Step 3 - optional: Set up networking, database, and tags', 'Step 4 - optional: Configure instance traffic and scaling', 'Step 5 - optional: Configure updates, monitoring, and logging', and 'Step 6: Review'. The main configuration area has two sections: 'Environment tier' with radio buttons for 'Web server environment' (selected) and 'Worker environment'; and 'Application information' with a text input for 'Application name' (containing 'MyWebApp') and an 'Application tags (optional)' section.

Beanstalk Configuration: Environment Tier for Dev Environment

aws Elastic Beanstalk Create environment

Environment information [Info](#)

Choose the name, subdomain and description for your environment. These cannot be changed later.

Environment name

MyWebApp-dev-env

Must be from 4 to 40 characters in length. The name can contain only letters, numbers, and hyphens. It can't start or end with a hyphen. This name must be unique within a region in your account.

Domain

Leave blank for autogenerated value .ap-south-1.elasticbeanstalk.com [Check availability](#)

Environment description

Platform [Info](#)

Platform

Node.js

Platform branch

Node.js 24 running on 64bit Amazon Linux 2023

Platform version

6.11.2 (Recommended)

Beanstalk Configuration: Environment Name & Platform for Dev Environment

aws Elastic Beanstalk Create environment

Application code [Info](#)

☐ Sample application

☐ Existing version

Application versions that you have uploaded.

☒ Upload your code

Upload a source bundle from your computer or copy one from Amazon S3.

Version label

Unique name for this version of your application code.

nodejs-v2-blue-dev

Source code origin. Maximum size 500 MB

☒ Local file

Upload application

[Choose file](#)

File name: nodejs-v2-blue.zip

File must be less than 500MB max file size

☐ Public S3 URL

Presets [Info](#)

Start from a preset that matches your use case or choose custom configuration to unset recommended values and use the service's default values.

Configuration presets

☒ Single instance (free tier eligible)

Beanstalk Configuration: Application Code, Version Label & Presets for Dev Environment

- Version label is a unique identifier for the application code uploaded to a beanstalk environment.

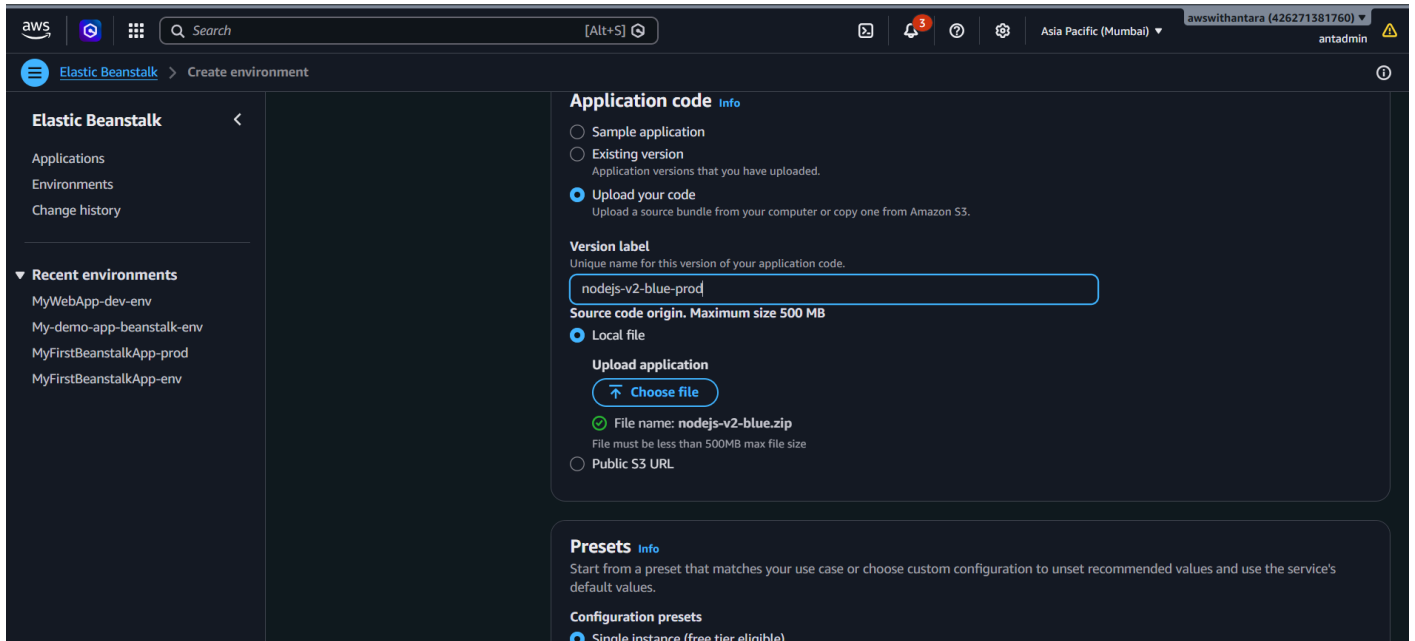
The screenshot shows the 'Configure service access' step in the AWS Elastic Beanstalk console. The left sidebar lists 'Elastic Beanstalk' with sub-items 'Applications', 'Environments', and 'Change history'. Below this is a 'Recent environments' section listing several environments. The main content area shows a progress bar with six steps: 'Step 1: Configure environment', 'Step 2: Configure service access' (current step), 'Step 3: optional: Set up networking, database, and tags', 'Step 4: optional: Configure instance traffic and scaling', 'Step 5: optional: Configure updates, monitoring, and logging', and 'Step 6: Review'. The 'Configure service access' section includes three main parts: 'Service access' (explaining IAM roles), 'Service role' (with a dropdown set to 'aws-elasticbeanstalk-service-role' and a 'Create role' button), 'EC2 instance profile' (with a dropdown set to 'aws-elasticbeanstalk-ec2-role' and a 'Create role' button), and 'EC2 key pair - optional' (with a dropdown set to 'Choose a key pair' and a 'Create key pair' button). At the bottom are 'Cancel', 'Skip to Review', 'Previous', and 'Next' buttons.

Beanstalk Configuration: Service Access for Dev Environment

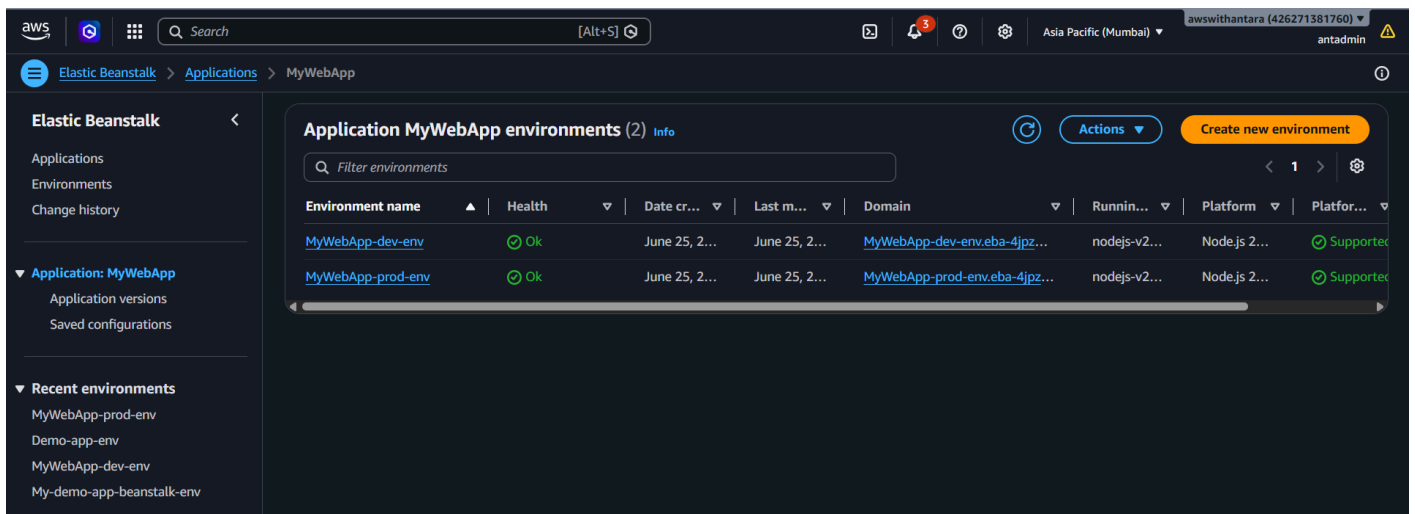
The screenshot shows the 'Environment information' step in the AWS Elastic Beanstalk console. The left sidebar is identical to the previous screenshot. The main content area shows the 'Environment information' section with a description: 'Choose the name, subdomain and description for your environment. These cannot be changed later.' It includes an 'Environment name' field with the value 'MyWebApp-prod-env' and a note: 'Must be from 4 to 40 characters in length. The name can contain only letters, numbers, and hyphens. It can't start or end with a hyphen. This name must be unique within a region in your account.' Below this is a 'Domain' field with the value '.ap-south-1.elasticbeanstalk.com' and a 'Check availability' button. The 'Environment description' section is currently empty. The 'Platform' section includes three dropdowns: 'Platform' set to 'Node.js', 'Platform branch' set to 'Node.js 24 running on 64bit Amazon Linux 2023', and 'Platform version' set to '6.11.2 (Recommended)'. At the bottom are 'Cancel', 'Skip to Review', 'Previous', and 'Next' buttons.

Beanstalk Configuration: Environment Name & Platform for Prod Environment

- Service role is used in an environment to allow beanstalk to perform adequate operations to maintain the environment
- EC2 instance profile is used in an environment to allow the underlying EC2 instances to perform required operations.



Beanstalk Configuration: Application Code, Version Label & Presets for Prod Environment



Beanstalk Console: Dev & Prod Environments

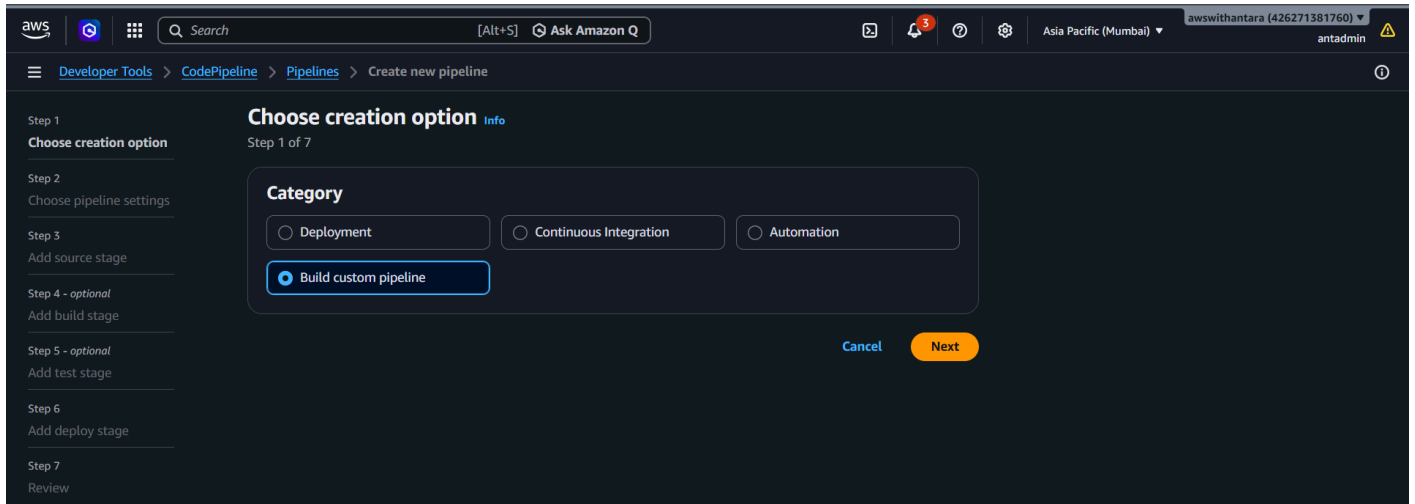
The Beanstalk application (**MyWebApp**) has two environments, dev and prod for CodePipeline requirements where the dev environment (**MyWebApp-dev-env**) serves as a testing stage to check whether the application is deployed properly with the desired functionalities, on the basis of which the final deployment happens in the prod environment (**MyWebApp-prod-env**) after a manual approval via setting up an action group in the prod environment stage in the pipeline.

Phase 4: The CI/CD Automator (AWS CodePipeline)

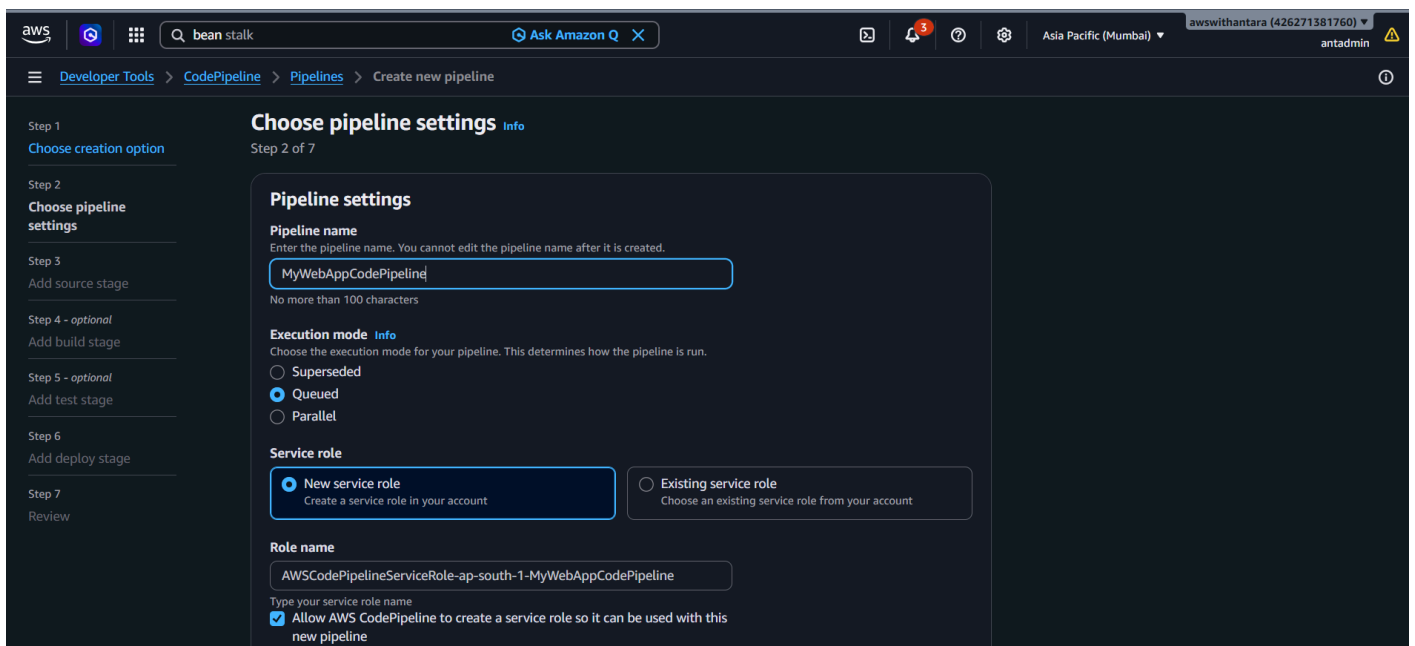
CodePipeline comprises a set of tools fully managed by AWS to automate the entire process of application code being pushed, built, tested, and deployed thereby eliminating the need to these manually every time code is modified.

CodePipeline Configuration:

- Category: Build custom pipeline
- Pipeline settings:
 - Pipeline name: MyWebAppCodePipeline
 - Execution mode: queued
 - Service role: New service role
 - Role name: AWSCodePipelineServiceRole-ap-south-1-MyWebAppCodePipeline
- Add source stage:
 - Source provider: GitHub (via GitHub App)
 - Connection: GithubConnectCodePipeline (connection established in configuration)
 - Repository name: ghubant2002/my-nodejs-app
 - Default branch: main
 - Output artifact format: CodePipeline default
 - Webhook: Start your pipeline on push and pull request events (to be enabled)
- Add build stage:
 - Build provider: AWS CodeBuild
 - Project name: spa-build-test (CodeBuild Project)
 - Build type: Single build
 - Region: Asia Pacific (Mumbai)
 - Input artifacts: SourceArtifact
- Add deploy stage:
 - Deploy provider: AWS Elastic Beanstalk
 - Region: Asia Pacific (Mumbai)
 - Input artifacts: BuildArtifact
 - Application name: MyWebApp
 - Environment name: MyWebApp-dev-env
 - Configure automatic rollback on stage failure: Enabled by default (recommended)
- Edit Permissions:
 - Add the policy **AdministratorAccess-AWSElasticBeanstalk** to the code pipeline (**MyWebAppCodePipeline**) service role to allow the pipeline to deploy the application to the beanstalk environment (**mywebapp-dev-env**)



CodePipeline Configuration: Build custom pipeline



CodePipeline Configuration: Pipeline Name, Execution Mode & Service Role

- Execution mode is set to **Queued** which ensures that the stages in the pipeline are processed one by one in the exact order they are triggered.
- The Service role is created by the CodePipeline console automatically.

The screenshot shows the AWS CodePipeline console interface. The top navigation bar includes the AWS logo, a search bar with 'bean stalk', and a user profile 'antadmin'. The breadcrumb trail is 'Developer Tools > CodePipeline > Pipelines > Create new pipeline'. The left sidebar lists steps from Step 1 to Step 7, with 'Add source stage' highlighted under Step 3. The main content area is titled 'Add source stage' and 'Step 3 of 7'. It contains the following configuration fields:

- Source**: Section header.
- Source provider**: A dropdown menu set to 'GitHub (via GitHub App)'.
- Connection**: A section with a text input 'arn:aws:codeconnections:ap-...' and a 'Connect to GitHub' button.
- Repository name**: A text input containing 'ghubant2002/my-nodejs-app'.
- Default branch**: A text input containing 'main'.

CodePipeline Configuration: Source Provider, Connection, Repository Name & Default Branch

The screenshot shows the 'Output artifact format' and 'Webhook events' configuration steps in the AWS CodePipeline console. The 'Output artifact format' section has two radio buttons: 'CodePipeline default' (selected) and 'Full clone'. Below this is a checkbox 'Enable automatic retry on stage failure' which is checked. The 'Webhook events' section has a checkbox 'Start your pipeline on push and pull request events' which is checked. There is also a 'Webhook event filters - optional' section with a 'Remove filters' button. At the bottom, there are 'Cancel', 'Previous', and 'Next' buttons.

CodePipeline Configuration: Output Artifact Format & Webhook Events

- A GitHub connection is established to link the source code repository to the pipeline.
- The webhook filter must be disabled in the build project (**spa-build-test**) in CodeBuild console, since it's enabled in the pipeline (**MyWebAppCodePipeline**) to trigger it upon modifications in GitHub code repository (**my-nodsjs-app**)

aws bean stalk Ask Amazon Q 3 ? ? Asia Pacific (Mumbai) awswithantara (426271381760) antadmin

Developer Tools > CodePipeline > Pipelines > Create new pipeline

Step 1
Choose creation option

Step 2
Choose pipeline settings

Step 3
Add source stage

Step 4 - optional
Add build stage

Step 5 - optional
Add test stage

Step 6
Add deploy stage

Step 7
Review

Add build stage - optional Info

Step 4 of 7

Build - optional

Build provider
Choose the tool you want to use to run build commands and specify artifacts for your build action.

☐ Commands ☒ Other build providers

AWS CodeBuild

Project name
Choose a build project that you have already created in the AWS CodeBuild console. Or create a build project in the AWS CodeBuild console and then return to this task.

Q spa-build-test X or [Create project](#)

☐ **Define buildspec override - optional**
Buildspec file or definition that overrides the latest one defined in the build project, for this build only.

CodePipeline Configuration: Build Provider & Project Name

Environment variables - optional Info

Choose the key, value, and type for your CodeBuild environment variables. In the value field, you can reference variables generated by CodePipeline. [Learn more](#)

[Add environment variable](#)

Build type

☒ **Single build**
Triggers a single build.

☐ **Batch build**
Triggers multiple builds as a single execution.

Region

Asia Pacific (Mumbai)

Input artifacts
Choose an input artifact for this action. [Learn more](#)

SourceArtifact X
Defined by: Source

☒ **Enable automatic retry on stage failure**

[Cancel](#) [Previous](#) [Skip build stage](#) [Next](#)

CodePipeline Configuration: Build Type, Region & Input Artifacts

- **SourceArtifact** is the packaged version of the application's raw files created by the very first stage, i.e., source stage
- The test stage is skipped in the pipeline (**MyWebAppCodePipeline**) configuration since build and test happens in the same CodeBuild Project (**spa-build-test**)

Add deploy stage Info
Step 6 of 7

Deploy - optional

Deploy provider
Choose how you want to deploy your application or content. Choose the provider, and then provide the configuration details for that provider.

Region
Asia Pacific (Mumbai)

Input artifacts
Choose an input artifact for this action. [Learn more](#)

BuildArtifact Defined by: Build

No more than 100 characters

CodePipeline Configuration: Deploy Provider, Region & Input Artifacts

Application name
Choose an application that you have already created in the AWS Elastic Beanstalk console. Or create an application in the AWS Elastic Beanstalk console and then return to this task.

Environment name
Choose an environment that you have already created in the AWS Elastic Beanstalk console. Or create an environment in the AWS Elastic Beanstalk console and then return to this task.

☒ Configure automatic rollback on stage failure

☐ Enable automatic retry on stage failure

[Cancel](#) [Previous](#) [Skip deploy stage](#) [Next](#)

CodePipeline Configuration: Application & Environment Names for Deploy Stage

Attach policy to AWSCodePipelineServiceRole-ap-south-1-MyWebAppCodePipeline

Current permissions policies (3)

Other permissions policies (1/1230)

Search:

Filter by Type: All types

☐	Policy name	Type	Description
☐	AccountManagementFromVercel	AWS managed	For use with accounts created through ...
☐	AdministratorAccess	AWS managed - job function	Provides full access to AWS services an...
☐	AdministratorAccess-Amplify	AWS managed	Grants account administrative permissi...
☒	AdministratorAccess-AWSElasticBeanstalk	AWS managed	Grants account administrative permissi...
☐	AIDevOpsAgentAccessPolicy	AWS managed	Provides permissions required by the A...
☐	AIDevOpsAgentActionsPolicy	AWS managed	Provides elevated permissions require...
☐	AIDevOpsAgentFullAccess	AWS managed	Provides full access to Amazon DevOp...
☐	AIDevOpsAgentReadOnlyAccess	AWS managed	Provides read only access to Amazon D...

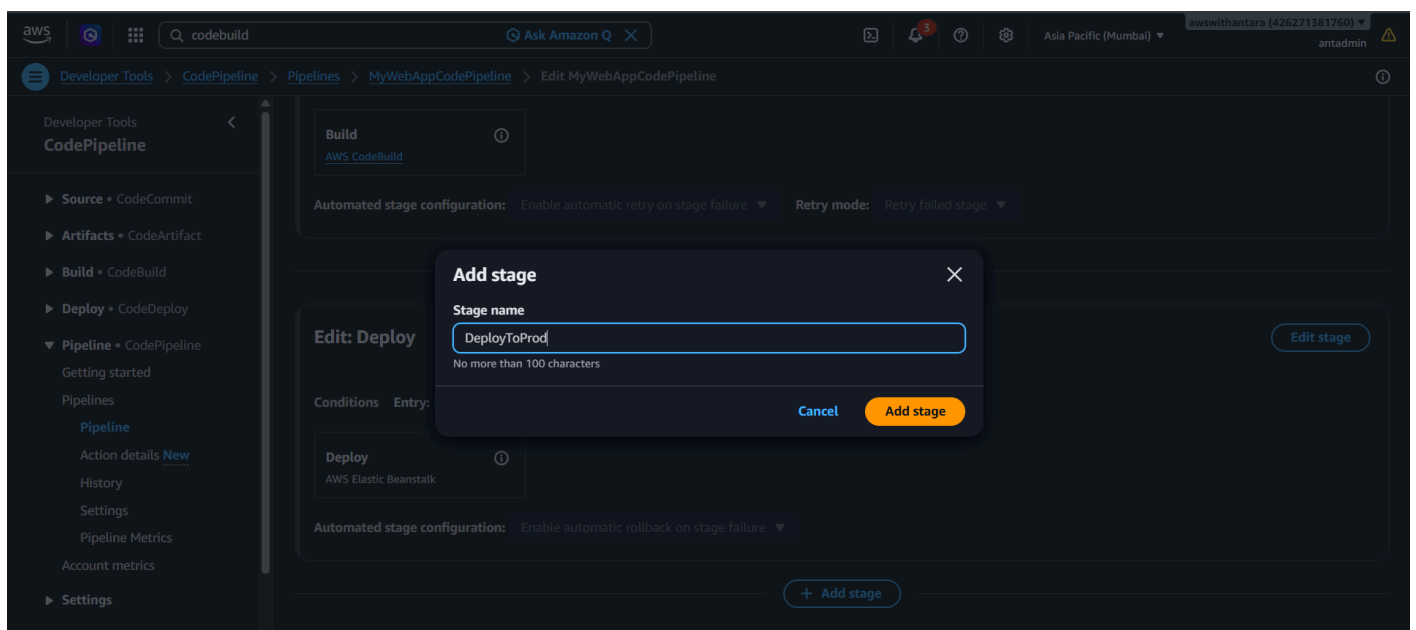
CodePipeline Configuration: Additional IAM Policy Attached to MyWebAppCodePipeline

BuildArtifact is the Input Artifact for the deploy stage which packages the application files excluding **buildspec.yml** as per explicit instructions in this file.

Multi-Environment Release Strategy

To ensure high availability and prevent deployment regressions, this pipeline utilizes a multi-environment deployment strategy:

- **Staging:** The isolated application artifact (**BuildArtifact**) is first deployed to the dev environment (**mywebapp-dev-env**). This allows pre-production validation, API testing, and UI/UX reviews in a safe sandbox.
- **Production:** Upon successful validation, the artifact (**BuildArtifact**) is promoted to the prod environment (**mywebapp-prod-env**), ensuring zero downtime and proper deployment.



CodePipeline Configuration: Add Production Stage

The screenshot shows the 'Edit action' configuration in the AWS CodePipeline console. The action is named 'DeployProdBeanstalk' and is provided by 'AWS Elastic Beanstalk'. The region is set to 'Asia Pacific (Mumbai)'. Under 'Input artifacts', 'BuildArtifact' is selected. The 'Application name' is 'MyWebApp' and the 'Environment name' is 'MyWebApp-prod-env'. The interface includes a sidebar with navigation icons, a top bar with the AWS logo, and a bottom bar with 'Cancel' and 'Done' buttons.

Edit action

Action name
Choose a name for your action
DeployProdBeanstalk
No more than 100 characters

Action provider
AWS Elastic Beanstalk

Region
Asia Pacific (Mumbai)

Input artifacts
Choose an input artifact for this action. [Learn more](#)
BuildArtifact
Defined by: Build
No more than 100 characters

Application name
Choose an application that you have already created in the AWS Elastic Beanstalk console. Or create an application in the AWS Elastic Beanstalk console and then return to this task.
MyWebApp

Environment name
Choose an environment that you have already created in the AWS Elastic Beanstalk console. Or create an environment in the AWS Elastic Beanstalk console and then return to this task.
MyWebApp-prod-env

Cancel Done

CodePipeline Configuration: Prod Stage Details

The screenshot shows the 'Edit action' configuration for a manual approval action. The action is named 'DeployToProdApproval' and is provided by 'Manual approval'. The interface includes a sidebar with navigation icons, a top bar with the AWS logo, and a bottom bar with 'Cancel' and 'Done' buttons.

Edit action

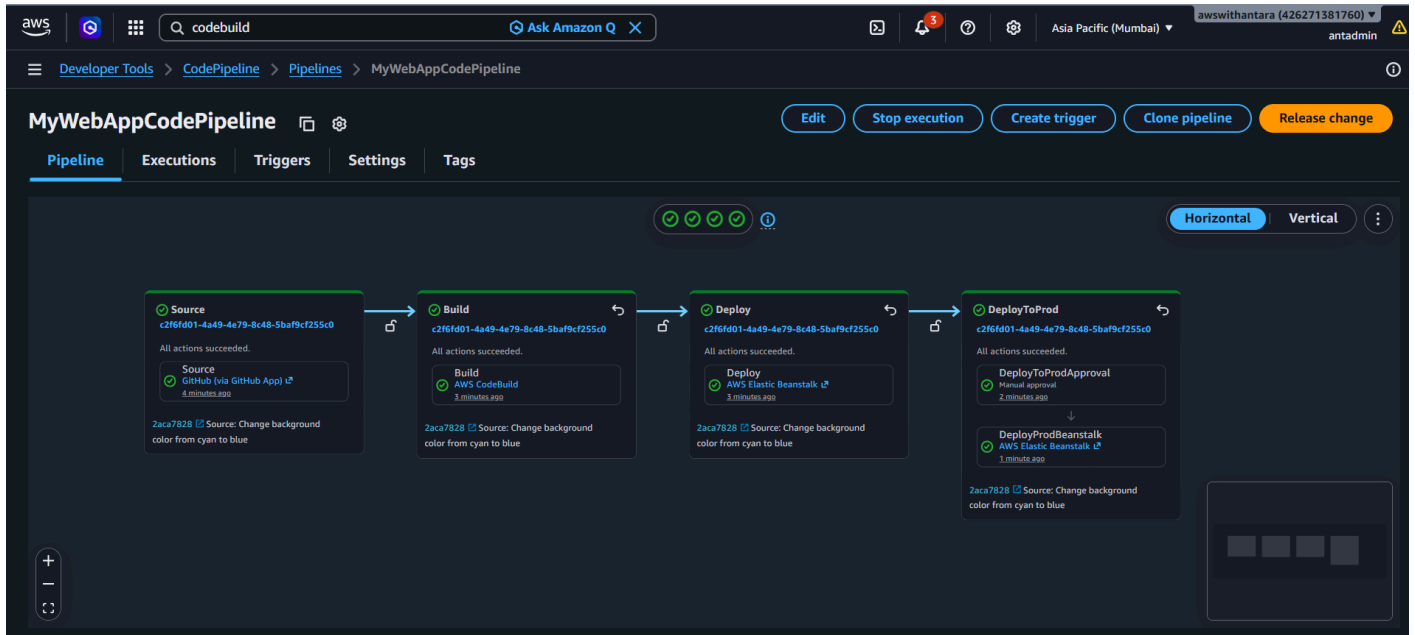
Action name
Choose a name for your action
DeployToProdApproval
No more than 100 characters

Action provider
Manual approval

Cancel Done

CodePipeline Configuration: Manual Approval for Prod Stage

The final deployment happens in the prod environment (**mywebapp-prod-env**) after a manual approval set up via an action group in the prod stage (**DeployToProd**) set up in the pipeline.



CodePipeline Console: Pipeline Stages & Status After Final Deployment

Inference: The implementation of this multi-stage CI/CD pipeline for a custom web application demonstrates infrastructure automation and continuous delivery. By utilizing AWS CodePipeline to seamlessly orchestrate GitHub, CodeBuild, and Elastic Beanstalk, this architecture eliminates manual deployment bottlenecks during release cycles.

- **Key Engineering Decisions:**

- **Artifact Isolation:** Explicitly configured the **buildspec.yml** artifacts block to isolate application code (**app.js**, **index.html**, **package.json**, **cron.yaml**). This ensures the final deployment package sent to Elastic Beanstalk remains lightweight and free of build specific configuration files.
- **Managed Compute:** Utilized Elastic Beanstalk to abstract the underlying EC2 instance provisioning and health monitoring, allowing the pipeline to focus strictly on application delivery.
- **Immutable Artifact Management:** Implemented a strict "Build Once, Deploy Many" artifact strategy. By configuring downstream pipeline stages to consume the exact same **BuildArtifact** generated by the build phase, the pipeline guarantees that the exact codebase validated in the testing phase is seamlessly promoted across Staging and Production environments without application bottlenecks.